PRACTICAL 11-A

Object Oriented Programming

AIM:

To implement and understand the concepts specified in Practical 11 of the CEUE203 Object Oriented Programming practical list, using Java. The work covers Part A: File Persistence and Report and Part B: Load/save account data and report.

CODE SCREENSHOT:

AccountFileStore.java 1

Part A > Practical 11-A > AccountFileStore.java > ...

```java
import java.io.*;
import java.nio.file.*;
import java.util.*;

public class AccountFileStore {
    Run | Debug
    public static void main(String[] args) throws Exception {
        Path p = Path.of("accounts.csv");
        List<String> rows = List.of("AC0001,Anshul,5000", "AC0002,Riya,2500");
        Files.write(p, rows);
        List<String> loaded = Files.readAllLines(p);
        System.out.println("Persistence verified: " + rows.equals(loaded));
        loaded.forEach(System.out::println);
    }
}

```

Code Screenshot 1

BankReport.java

Part A > Practical 11-A > BankReport.java > ...

```java
import java.util.*;

public class BankReport {
    record Tx(String type, long amount) {
    }

    Run | Debug
    public static void main(String[] args) {
        List<Tx> tx = List.of(new Tx(type: "DEPOSIT", amount: 5000), new Tx(type: "WITHDRAW", amount: 1200), new Tx(type: "DEPOSIT", amount: 300));
        long deposits = tx.stream().filter(t -> t.type.equals("DEPOSIT")).mapToLong(Tx::amount).sum();
        long withdrawals = tx.stream().filter(t -> t.type.equals("WITHDRAW")).mapToLong(Tx::amount).sum();
        System.out.println("Transactions: " + tx.size());
        System.out.println("Deposits: " + deposits);
        System.out.println("Withdrawals: " + withdrawals);
        System.out.println("Net: " + (deposits - withdrawals));
    }
}

```

Code Screenshot 2

PersistenceDemo.java

Part A > Practical 11-A > PersistenceDemo.java > ...

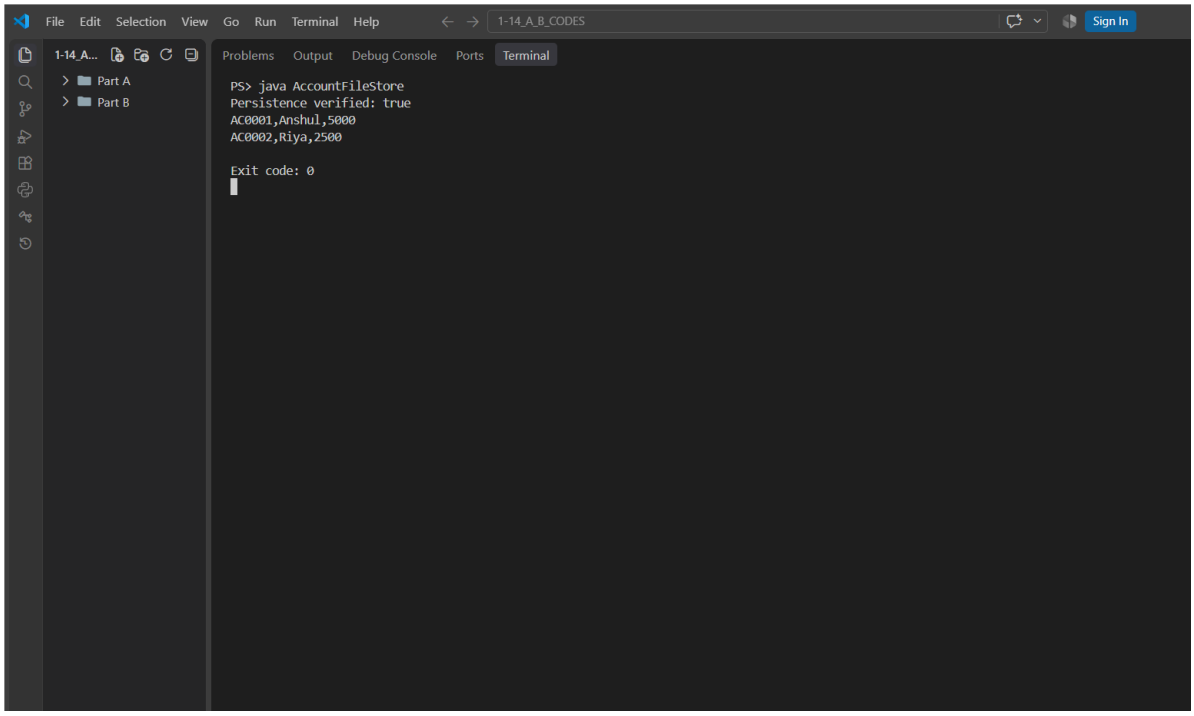
```java
import java.io.*;
import java.nio.file.*;
import java.util.*;

public class PersistenceDemo {
    Run | Debug
    public static void main(String[] args) throws IOException {
        Path path = Path.of("accounts.txt");
        List<String> accounts = Files.exists(path) ? Files.readAllLines(path) : new ArrayList<>();
        if (accounts.isEmpty())
            accounts.add("AC0001,Anshul,5000");
        Files.write(path, accounts);
        System.out.println("Loaded accounts:");
        accounts.forEach(System.out::println);
        System.out.println("Saved to " + path.toAbsolutePath());
    }
}

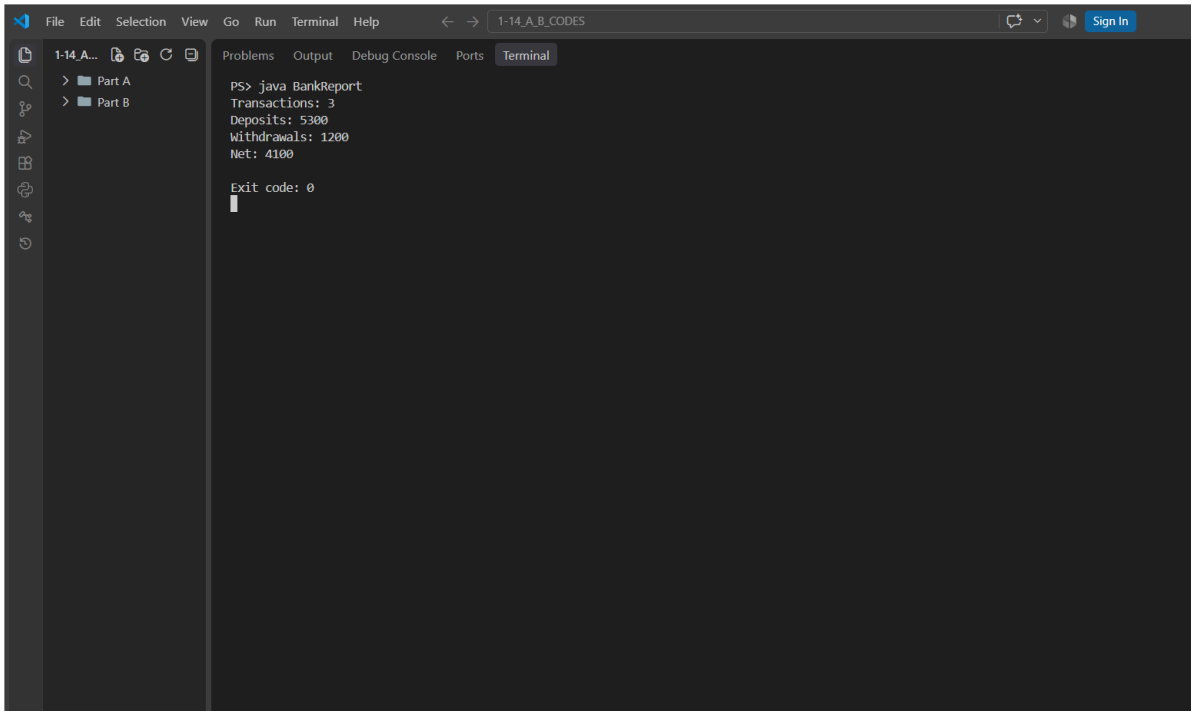
```

Code Screenshot 3

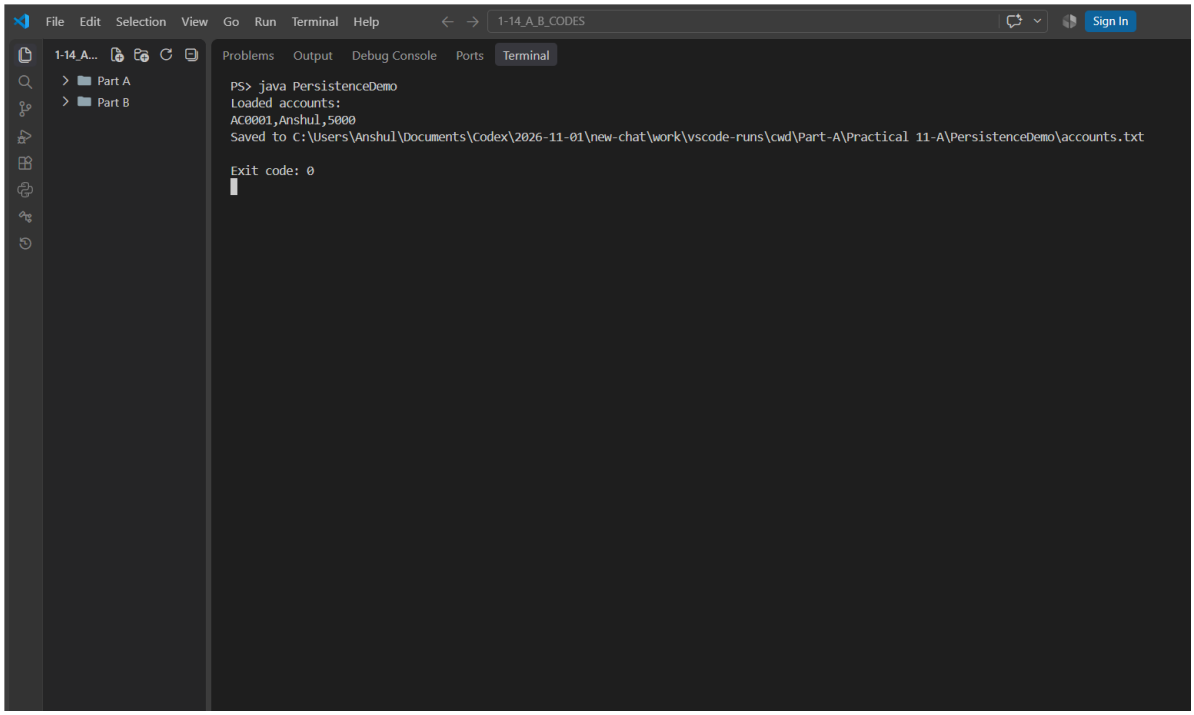
OUTPUT SCREENSHOT:



Output Screenshot 1



Output Screenshot 2



Output Screenshot 3

LEARNING OUTCOMES:

Persist application data to files, reload saved records, and generate a basic report.